Zraim Uichi R. Ebrada Mobile Development Mock Midterm

Milestone 1 – UI Grid & Styling

- I used ConstraintLayout as the root in activity_main.xml and gave it a dark background so the app looks like a custom dark-themed calculator.

- I added a TextView named textDisplay at the top, set to 0dp width/height with constraints and a height percent so it takes around 20% of the screen, with big, white, right-aligned text on a slightly lighter

- Under the display, I placed a GridLayout with 4 columns and 5 rows to act as the keypad area for all my buttons.

- I arranged the buttons in a normal calculator order, the clear and the custom operator on the top row, then digits 0–9, the decimal point, and the operators +, -, ×, /, = at the right sidemost of the grid.

- I used shared styles for number and operator buttons, and I pulled all labels (digits, operators, and the custom “×0.83” text) from strings.xml so nothing is hardcoded and it’s easier to maintain or localize later.

Milestone 2 – Core Math Logic & Edge Cases

- For Milestone 2, I focused on making the calculator actually work and making sure errors, especially division by zero, are handled safely and effectively.

- My MainActivity extends AppCompatActivity and implements View.OnClickListener, which means I can handle all button clicks inside one onClick method and just route them to the right helper functions.

- I keep track of three numbers: num1 for the first operand, num2 for the second operand when the user presses =, and num3 for the last result; I also store the current operator in activeOperator and use two flags (isOperatorSelected and isErrorState) to know if I’m waiting for the second operand or if the calculator is in an error state.

- In onCreate, I loop through all digit button IDs and attach the same click listener, then I also hook up the dot, the four operators, equals, clear, and the custom button to the same listener so all input goes through the same path.

- Inside onClick, I first check if the calculator is in an error state; if it is, I ignore everything except Clear; then I branch: Clear calls clearAll, operator buttons call onOperatorClicked, = calls onEqualClicked, the custom button calls onCustomOperatorClicked, and all other

buttons are treated as digits or the decimal and go to onDigitOrDotClicked.

- onDigitOrDotClicked builds the current input as a string, resets it when I'm starting a new number (after "0", after selecting an operator, or after an error), and blocks any extra decimal point when there's already one in the current value.

- onOperatorClicked reads the current display, skips invalid or error text, parses it into num1, sets activeOperator based on which operator button was pressed, and marks isOperatorSelected = true so the next digits belong to the second operand.

- onEqualClicked only runs if there is an active operator and some text on the display; it parses num2, calls calculate(num1, num2, activeOperator), updates the display with the result, and if there is no error it also stores that result into num3 before resetting the operator state.

- calculate(first, second, op) is where the math happens; it has a special case for division where, if the operator is / and second is 0.0, it shows "Cannot divide by zero", sets isErrorState = true, and returns that message; otherwise, it performs the correct arithmetic (+, −, ×, ÷) and returns the result as a string and clears the error flag.

- clearAll() resets all numeric variables, clears the operator and flags, and puts the display back to "0" so the user can start fresh at any time.

Milestone 3 – State Preservation & Custom Operator

- In Milestone 3, my main goal was to make sure the calculator does not reset when the screen rotates, so the current numbers and operator are still there after configuration changes.

- I defined keys for every important piece of data I want to keep: the display text (KEY_DISPLAY), the three numeric values (KEY_NUM1, KEY_NUM2, KEY_NUM3), the active operator (KEY_OPERATOR), and the two flags for operator selection and error state (KEY_OPERATOR_SELECTED, KEY_ERROR_STATE).

- In onSaveInstanceState, I store the current display value, all three operands, the active operator string, and both flags into the Bundle using those keys; this method runs automatically before Android destroys the activity, for example during rotation.

- I kept my restoreState method to do the opposite: it reads the saved values back from the Bundle, assigns them to my member variables, and updates the textDisplay text so the UI matches the saved state.

- In onCreate, I check if savedInstanceState is not null; if it has data, I call restoreState(savedInstanceState), which allows the calculator to reopen with the same

operands, operator, display text, and error status instead of going back to zero after every rotation.